Learnable Hodge–Laplacian Driven Mixture of Experts for Steady-PDE Surrogates on Coarse Unstructured Meshes

Stavros Nousias[1], **Akis Nousias**[2], **Konstantinos Gkrispanis**[1]

[1]Technical University of Munich
[2]K3Y Labs

Abstract

Surrogate models approximate the solutions of partial differential equations given a certain geometry and boundary conditions. Traditional transformer-based approaches employ a single mixing mechanism to the whole solution. although the solution fields exhibit variations across multiple spatial scales. We propose a mixture of transformer experts that combines a routing mechanism with multiscale supervision. Specifically, the individual experts are routed through a controller that utilizes learnable Laplacian matrices to compute weights that are subsequently used to define a weighted sum of the experts for the output feature vectors at each point of the domain. Subsequently, a decoder receives the weighted sum of the experts output and predicts the solution fields. During training, the reference fields are decomposed into components at different spatial scales by repeated smoothing with a mesh Laplacian. We evaluate our approach on the official AirFRANS dataset using remeshed solution domains leading to unstructured triangular meshes of 4.9k vertices. The experimental outcomes reveal lower mean relative L_2 error and interior mean squared errors among the compared models of this study. Code and data are publicly available in https://github.com/snousias/hodge-moe-pde.

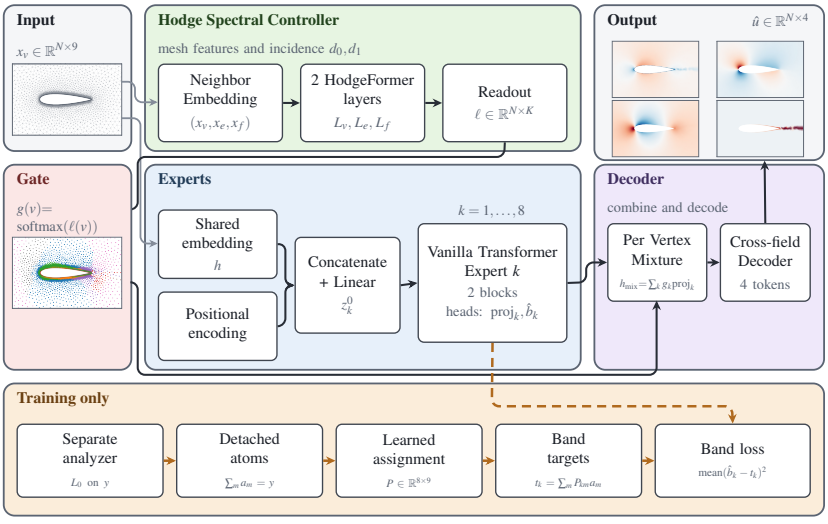


Figure 1: Model architecture overview. The controller routes transformer experts that combine shared embeddings with positional encodings. Their representations are mixed and decoded into four fields. During training, a separate analyzer decomposes the reference fields into atoms, which a learned assignment combines into targets for the auxiliary expert heads (dashed path).

Preprint.

1 Introduction

Computational fluid dynamics is central to aerodynamic design. Steady Reynolds-averaged Navier–Stokes (RANS) simulations provide pressure, velocity and turbulence fields for a candidate geometry [1]. Their computational cost motivates learned models for aerodynamic prediction [2, 3].

Learned surrogates predict solution fields from geometry, boundary data and flow parameters, reducing inference cost relative to the reference solver [4]. Physics-informed networks train with penalties on equation residuals and boundary-condition errors [5, 6]. For RANS, evaluating these residuals also requires a representation of the Reynolds stresses [3]. Neural operators learn maps between functions using parameters shared across discretisations [7, 8]. The Fourier neural operator represents global interactions through learned Fourier multipliers [9]. On irregular domains, graph kernel networks aggregate over sampled neighbourhoods [10], Geo-FNO learns a deformation to a computational domain for Fourier processing [11], and MeshGraphNets update features along mesh-based interaction graphs [12].

Transformer operators use linear attention or factorized kernels to reduce the cost of dense attention [13–16]. Transolver aggregates point features into learned slice tokens, applies attention between those tokens, and distributes the updated features back to the points [17]. For a fixed number of tokens and fixed feature width, its cost grows linearly with the number of input points.

A separate line of work uses mesh differential operators based on discrete exterior calculus [18]. HodgeNet learns diagonal or block-diagonal Hodge stars and constructs features from the resulting Laplacian spectrum [19]. DiffusionNet learns diffusion times for feature propagation using geometric Laplacians [20]. HodgeFormer parameterizes Hodge stars through attention on vertices, edges and faces [21]. These methods use differential operators within the prediction network.

Sparse mixtures of experts increase parameter capacity while evaluating only selected experts for each input [22, 23]. Geometric weighting is also used in PDE operator learning. GNOT computes weights for feedforward experts from query coordinates [15] and GAOT combines graph-operator features from several neighbourhood radii using coordinate-dependent softmax weights at latent encoder points [24].

In this work, we use a learnable Laplacian controller to combine plain transformer experts at each mesh vertex. Auxiliary targets constructed from the reference field encourage the experts to represent different scales. A second, diagonal-star Hodge Laplacian decomposes the training target into a sequence of components. The contributions of this paper can be summarized as follows: (i) We discuss a Gaussian star construction (section 3.4) that is symmetric positive definite (ii) We propose an architecture that uses the learned discrete Hodge Laplacian to compute routing weights. (iii) We introduce a supervision scheme in which a second, learnable Laplacian decomposes the target into a sequence of components (section 3.6). (iv) We evaluate our approach on the official AirFRANS split against a Transolver baseline (section 4.2). Section S1 lists every symbol used below, with its type and its meaning. The remainder of this paper is organized as follows: section 2 reviews related work, section 3 presents the method, section 4 describes the simulation setup and reports the results, and section 5 concludes this paper.

2 Related Work

Physics-informed and data-driven surrogates.

Physics-informed neural networks approximate PDE solutions by penalizing equation residuals and boundary-condition errors during training [5, 6]. The Reynolds-averaged Navier–Stokes (RANS) equations contain Reynolds stresses that represent the effects of velocity fluctuations. These terms require a closure relation, which can be specified by a turbulence model or learned from data [3, 2].

Supervised flow surrogates learn from reference solutions to predict fields from geometry and flow conditions. Their training objective uses field-prediction errors rather than a governing-equation residual [4].

Neural operators on irregular domains.

Neural operators constitute a generalization of neural networks to learn maps between infinite-dimensional function spaces [7, 8]. Seminal works include the DeepONet and Fourier neural

operator (FNO). The former encodes sampled input-function values in a branch network and output query locations in a trunk network [25], while the latter represents nonlocal interactions through learned multipliers in the Fourier domain [9].

Irregular meshes or varying geometries require further choices on how to represent the underlying domain. Graph kernel networks evaluate learned integral kernels over sampled neighbourhoods [10], while Geo-FNO learns a deformation between the physical domain and a latent computational domain [11]. GINO uses graph-based mappings to move features onto a regular latent grid and return them to the output locations, where an FNO processes them on that grid [26]. Finally, MeshGraphNets pass features repeatedly along mesh-based interaction graphs [12].

Transformer PDE solvers.

Dense self-attention exhibits quadratic computational complexity with respect to token count. Transformer PDE surrogates use several approximations to reduce this cost. Galerkin attention uses a linearised formulation without softmax [13] and OFormer combines self-attention with cross-attention for operator learning [14], while GNOT uses heterogeneous normalised attention to handle multiple inputs on irregular discretisations [15]. On multidimensional grids, FactFormer reduces interaction costs by factorizing the attention kernel along the coordinate axes [16].

Wu et al. [17] proposed the Transolver architecture. The model first aggregates pointwise features into a prescribed number of learned slices referred to as slice-tokens. Attention operates on these tokens, after which the updated information is distributed back to the points [17]. For a fixed feature size and number of slices, the attention cost grows linearly with the number of mesh points. In this work, we use Transolver as our principal comparison baseline.

Geometric gating and multiscale mixtures.

Mixture-of-experts architectures use learned routing to combine specialized subnetworks. Sparse variants evaluate a subset of experts for each input, increasing parameter capacity without evaluating every expert [22, 23]. GNOT uses query coordinates to compute weights for feedforward experts within transformer blocks [15]. GAOT aggregates graph-operator features at several neighbourhood radii and combines them using coordinate-dependent softmax weights at latent encoder points [24].

Our controller uses learned Laplacian operators to weight full transformer experts at each mesh vertex. Auxiliary expert outputs are trained against components of the reference field obtained through mesh-spectral filtering. The controller determines how expert outputs are combined, while the auxiliary targets encourage the experts to predict different scale-dependent components.

Learned differential operators and spectral analysis.

Discrete exterior calculus represents connectivity through incidence operators and metric-dependent quantities through discrete Hodge stars [18]. A k-cochain assigns values to oriented mesh elements of dimension k. Hodge Laplacians extend vertex-based graph Laplacians to these values, so they can act on edge and face signals as well as signals on higher-dimensional cells [27, 28].

Several learning methods use discrete differential operators. HodgeNet learns mesh spectral geometry through a data-driven construction of the operators [19]. DiffusionNet uses mesh differential operators to propagate features and learns the diffusion times that control that propagation [20]. DiffGCN expresses graph convolutions through discretized differential operators and uses algebraic-multigrid pooling [29]. HodgeFormer constructs Hodge operators through attention and applies them to features on mesh vertices, edges, and faces [21].

For PDE learning, Geo-NeW learns differential operators together with compatible reduced finite-element spaces [30]. Hodge Spectral Duality combines a low-frequency Hodge spectral branch, including harmonic modes, with a spatial branch for higher-frequency components [31]. Topological Neural Operators couple features on cells of different dimensions through discrete differential operators [32].

3 Methodology

3.1 Overview

We propose a mixture of transformer experts for steady-PDE prediction on unstructured meshes, with per-vertex weights computed by a learnable Laplacian controller. The model predicts velocity components u_x, u_y, pressure p, and turbulent kinematic viscosity ν_t at the vertices of an unstructured mesh. The prediction path consists of a shared embedding, a controller, transformer experts, and a field decoder, where the controller produces routing weights from vertex, edge, and face features together with the mesh incidence matrices and neighborhood indices. Each expert combines the shared embedding with fixed Fourier positional features and produces both a latent representation and an auxiliary prediction. The aforementioned controller uses two **PSD-Hodge layers**, based on the HodgeFormer [21] architecture.

3.2 Simplicial complex and neighborhoods

The mesh is the simplicial complex $\mathcal{M} = (V, E, F)$, with $N = |V|$, $N_e = |E|$, and $N_f = |F|$ and signed incidence matrices $d_0 \in \mathbb{R}^{N_e \times N}$ and $d_1 \in \mathbb{R}^{N_f \times N_e}$, with non zero entries -1 or $+1$, mapping vertex values to edges and edge values to faces, respectively. For each mesh element i, we identify its neighborhood as the set of elements of the same type whose features contribute at i. We call this set the *operator support* because it specifies the entries that may be nonzero in row i. The stored neighbor list is $\mathcal{N}^{\text{full}}(i)$, while $\mathcal{N}(i) \subseteq \mathcal{N}^{\text{full}}(i)$ denotes the subset used by the controller. Vertex and edge neighbor lists are initially generated from graph-hop rings.

3.3 Input features and input embeddings

We consider the input features $x_v \in \mathbb{R}^{N \times c_v}$, $x_e \in \mathbb{R}^{N_e \times c_e}$, and $x_f \in \mathbb{R}^{N_f \times c_f}$.

$$x_v^{\text{raw}} = [\text{pos}^{\text{raw}}1, \text{pos}^{\text{raw}}2, A_v^{\text{dual}}, u\text{in}, v\text{in}, \text{SDF}, n_x, n_y, \text{is}_{\text{w}}\text{all}], \tag{1}$$

where $\text{pos}^{\text{raw}}1, \text{pos}^{\text{raw}}2$ are physical coordinate columns, A_v^{dual} is the barycentric dual area, computed as one third of the sum of incident triangle areas, and SDF denotes signed distance. $u\text{in}, v\text{in}$ refer to inlet velocity components, n_x, n_y to normal vector and $\text{is}_{\text{w}}\text{all}$ indicates whether a vertex belongs to a wall segment.

For each input feature Z-score standardization is defined as

$$(x_v)ij = \frac{(x_v^{\text{raw}})ij - \mu_j^{\text{in}}}{s_j^{\text{in}} + 10^{-8}}, \qquad j = 1, \ldots, c_v. \tag{2}$$

,where μ_j^{in} and s_j^{in} are computed by sampling the training meshes and the constant value 10^{-8} prevents division by zero. The freestream speed is obtained from the inlet velocity components at the marginal vertices, and the dataset scales the velocity components u_x, u_y, the pressure p, and the turbulent kinematic viscosity ν_t:

$$U_\infty = \sqrt{u_{\text{in}}^2 + v_{\text{in}}^2}, \qquad \widetilde{y} = \left[\tfrac{u_x}{U_\infty}, \tfrac{u_y}{U_\infty}, \tfrac{p}{U_\infty^2}, \tfrac{\nu_t}{U_\infty}\right], \qquad y_{vc} = \frac{\widetilde{y}_{vc} - \mu_c^{\text{norm}}}{s_c^{\text{norm}} + 10^{-8}}. \tag{3}$$

,v corresponds to vertices, c to output channels, and μ_c^{norm}, s_c^{norm} are the mean and standard deviation of the scaled training targets for channel c, computed across vertices. μ_c^{norm}, s_c^{norm} are reused for all training and test cases. The standardized target is expressed as $y \in \mathbb{R}^{N \times 4}$ and the model output as $\hat{u} \in \mathbb{R}^{N \times 4}$.

The prediction branch maps the input features to input embeddings to one token per vertex through a shared two-layer perceptron with GELU activation. All eight experts receive the same token array h.

$$h = W_2 \text{GELU}(W_1 x_v) \in \mathbb{R}^{N \times d}, \qquad d = 192. \tag{4}$$

The controller uses a separate neighbor embedding for vertex, edge, and face features and then maps each stream through its own three-layer perceptron, with ReLU after every linear layer. The resulting streams are $h_v \in \mathbb{R}^{N \times d_c}$, $h_e \in \mathbb{R}^{N_e \times d_c}$, and $h_f \in \mathbb{R}^{N_f \times d_c}$. Layer normalization is applied inside the subsequent PSD-Hodge layers.

3.4 Hodge controller

Learned star operators. The learned operators follow the approach in HodgeFormer [21] that retains compositions of incidence and Hodge matrices while parameterizing the Hodge matrices through attention. Using the streams defined in section 3.3, each head first projects the normalized features to $\varphi_i = W_\varphi \mathrm{LN}(h_s)_i$, where $s \in v, e, f$ and $\varphi_i \in \mathbb{R}^{d_h}$.

The controller has embedding size $d_c = 64$ and number of heads $H = 4$ heads. The per-head kernel-feature dimension is $d_h = d_c/H = 16$.

These projected features determine the interaction between element i and each neighbor $j \in \mathcal{N}(i)$ through a Gaussian kernel:

$$w_{ij} = \exp\left(-\frac{|\varphi_i - \varphi_j|_2^2}{2\sigma^2}\right), \qquad \sigma = \mathrm{softplus}(\gamma) + 10^{-8}. \tag{5}$$

The matrix Ω assigns a weight to a position if the element belongs to the set of vertices comprising the effective support. We normalize the weights using the row sums $D_i = \sum_j \Omega_{ij}$, with $D = \mathrm{diag}(D_i)$, and combine the resulting operator with the identity:

$$\hat{S}_{ij} = \frac{\Omega_{ij}}{\sqrt{D_i D_j + 10^{-8}}}, \qquad \star x = (1-\beta)x + \beta \hat{S} x, \qquad \beta = \tfrac{1}{2}\,\mathrm{sigmoid}(\alpha). \tag{6}$$

The spectral properties of this construction depend on the neighborhood relation. A neighborhood is reciprocal when $j \in \mathcal{N}(i)$ implies $i \in \mathcal{N}(j)$. Under this condition, the ideal normalized matrix $D^{-1/2}\Omega D^{-1/2}$ is symmetric and has eigenvalues in $[-1, 1]$. The star formed with this ideal normalization is therefore symmetric positive-definite (SPD), with eigenvalues in $[1 - 2\beta, 1]$.

For the mesh faces, no neighborhoods are considered. The face star therefore uses a diagonal form:

$$z_i = \frac{|\varphi_i|_2^2}{d_h \sigma^2}, \qquad (\star_2 x)_i = (1 - \beta + \beta e^{-z_i}) x_i, \tag{7}$$

This star scales each face independently, and its diagonal remains positive. Stars labeled as inverse operators have their own learnable parameters and they are not calculated matrix inverses.

Operator composition. We denote the learned operators on vertices, edges, and faces by L_v, L_e, and L_f. For one attention head, let x_v, x_e, x_f denote the normalized latent features supplied to that head.

$$\begin{aligned} L_v &:= \star_0^{-1}(x_v) \cdot d_0^T \cdot \star_1(x_e) \cdot d_0, \\ L_e &:= d_0 \cdot \star_0^{-1}(x_v) \cdot d_0^T \cdot \star_1(x_e) + \star_1^{-1}(x_e) \cdot d_1^T \cdot \star_2(x_f) \cdot d_1, \\ L_f &:= d_1 \cdot \star_1^{-1}(x_e) \cdot d_1^T \cdot \star_2(x_f). \end{aligned} \tag{8}$$

The vertex operator reads vertex and edge features, the edge operator reads all three feature arrays, and the face operator reads edge and face features. $\star_0^{-1}$ and $\star_1^{-1}$ have separate learnable parameters. The two terms of L_e act on the same projected edge values and their outputs are added. The head outputs are concatenated and projected back to the embedding dimension, as described below.

If the stars in L_v are symmetric positive definite, then $Q = d_0^T \star_1 (x_e) d_0$ is symmetric positive semidefinite and

$$L_v = \star_0^{-1}(x_v) Q \sim \star_0^{-1/2}(x_v) Q \star_0^{-1/2}(x_v). \tag{9}$$

Here $\star_0^{-1/2}(x_v)$ denotes the symmetric square root of the independently learned $\star_0^{-1}(x_v)$. Under this assumption, L_v has a real, nonnegative spectrum, although it is generally not symmetric in the Euclidean inner product.

3.5 Transformer experts and decoder

Expert k receives the token array h joined with Fourier features of $\mathrm{pos} \in \mathbb{R}^{N\times 2}$, the two standardized coordinate columns of x_v:

$$\begin{aligned} z_k^0 &= \big[h \parallel \sin(2\pi\,\mathrm{pos} B_k) \parallel \cos(2\pi\,\mathrm{pos} B_k)\big](W_{\mathrm{in}}^k)^\top \in \mathbb{R}^{N\times d_x}, \\ B_k &= \sigma_k G_k \in \mathbb{R}^{2\times n_F}. \end{aligned} \tag{10}$$

Here $\|$ denotes concatenation along the feature dimension, and $W_{\text{in}}^k \in \mathbb{R}^{d_x \times (d+2n_F)}$ is the input projection weight. The expert width is $d_x = 88$, the number of frequencies is $n_F = 8$, and the entries of G_k are drawn once from a standard normal distribution and then held fixed. All projections in this subsection include bias terms, which are omitted from the equations. The fixed scales are $(\sigma_1, \ldots, \sigma_K) = (1.0, 1.7, 2.8, 4.8, 8.0, 13.5, 22.6, 38.0)$. The experts share the same architecture and have separate trainable parameters.

Each expert applies two transformer blocks to z_k^0 and each block uses multi-head softmax self-attention over all N vertices and a feedforward network, with layer normalization before each operation. A final layer normalization produces z_k. It is noteworthy that, the experts receive no incidence matrices and no neighborhood indices. Two linear maps read z_k: proj_k, which returns an array of width d, and W_b^k, which returns the auxiliary prediction $\hat{b}_k \in \mathbb{R}^{N \times 4}$. The auxiliary prediction is supervised against the per-expert target constructed in section 3.6 and is not used to form the field prediction.

The controller logits pass through a softmax, which gives weights $g(v) \in \Delta^K$ that are nonnegative and sum to one, and the expert representations are combined as

$$h_{\text{mix}}(v) = \sum_{k=1}^{K} g_k(v)\, \text{proj}_k(z_k)(v) \in \mathbb{R}^d. \tag{11}$$

The weights set the combination and not the number of experts evaluated, so routing does not reduce expert computation.

Subsequently, the decoder receives as input h_{mix} and applies layer normalization and dropout to, yiedling $m(v) \in \mathbb{R}^d$, and forms one token per output channel:

$$t_{vc}^0 = m(v) + E_{\text{field},c}, \qquad c = 1, \ldots, 4. \tag{12}$$

3.6 Auxiliary targets

One averaging step replaces each vertex value by a weighted mean of that value and the values at its neighbors. Repeating the step m times mixes each vertex with the vertices within m edges, which attenuates variation finer than m edges and leaves coarser variation almost unchanged. The reference field is averaged at eight increasing step counts, from two to thirty-two. The nine components are the most averaged field, the differences between consecutive averaged fields, and the difference between the reference field and the least averaged one.

Each expert is trained to reproduce an assigned weighted sum of the components, and the field prediction is trained against the reference fields rather than against the components. Additionally, each expert produces an auxiliary prediction $\hat{b}_k$ allowing to seperate the reference field is into components that vary over different distances on the mesh.

Conductances. A conductance is a parameter $w_e > 0$ assigned to edge e. Since the mesh cells differ in size, the number of edges between two vertices is not proportional to the distance between them, and averaging over the unweighted graph would cover different amounts of space. The conductances are computed by an edge network MLP that maps the c_e features of each edge to a scalar through a hidden layer with GELU activation, and softplus layer:

$$w_e = \text{softplus}(\text{MLP}(x_e)) + \epsilon_w, \qquad \epsilon_w = 10^{-4}. \tag{13}$$

The conductances then define two diagonal operators and a Laplacian:

$$\star_1 = \text{diag}(w), \qquad \mu = |d_0|^\top w, \qquad \star_0 = \text{diag}(\mu), \qquad L_0 = \star_0^{-1} d_0^\top \star_1 d_0. \tag{14}$$

where $\mu_v = \sum e \ni v w_e$ is the total conductance of the edges containing vertex v. We use signed incidence matrices with nonzero entries -1 and $+1$. For any positive conductances

$$x^\top d_0^\top \star_1 d_0 x = \sum e = (i,j) w_e (x_i - x_j)^2 \geq 0, \tag{15}$$

so L_0 is positive semidefinite by construction.

The solution at different spatial scales. Each smoothing step replaces each vertex value by the conductance-weighted average of its neighbors,

$$\bar{y}_i \leftarrow \tfrac{1}{2}\bar{y}_i + \tfrac{1}{2} \sum_{j \sim i} \frac{w_{ij}}{\mu_i} \bar{y}_j \tag{16}$$

which is $\bar{y} \leftarrow (I - \tau L_0)\bar{y}$ with $\tau = \frac{1}{2}$.

We retain the smoothed fields at $L = 8$ step counts,

$$(m_0, \ldots, m_{L-1}) = (2, 4, 6, 8, 12, 16, 24, 32), \qquad \bar{y}^{(j)} = (I - \tau L_0)^{m_j} y, \quad j = 0, \ldots, L-1, \tag{17}$$

and form $M = L + 1 = 9$ components. We call these components atoms:

$$a_0 = \bar{y}^{(L-1)}, \qquad a_i = \bar{y}^{(L-1-i)} - \bar{y}^{(L-i)} \quad (1 \le i \le L-1), \qquad a_{M-1} = y - \bar{y}^{(0)}. \tag{18}$$

Assignment. Given $M = 9$ atoms and $K = 8$ experts, the target of each expert is a weighted sum of the atoms. For each expert–atom pair we hold a score $\theta \in \mathbb{R}^{K \times M}$. A softmax function across experts converts the scores of each atom into nonnegative weights:

$$P_{km} = \frac{\exp(\theta_{km}/T)}{\sum_{k'=1}^{K} \exp(\theta_{k'm}/T)}, \qquad T = 1, \qquad t_k = \sum_{m=0}^{M-1} P_{km} a_m. \tag{19}$$

During training, the atoms are frozen when forming the targets. The training loss can therefore update the assignment scores without passing gradients through the atoms. The assignment can combine several atoms in one expert target or distribute one atom across several targets.

3.7 Training Objective

We train the final prediction to match the reference fields and each expert's auxiliary output to match its assigned target. Two additional penalties encourage balanced average routing weights and discourage assignments that give an expert a target with a very small share of the total squared target norm. The combined objective is

$$\mathcal{L} = \mathcal{L}_{\text{vol}} + \mathcal{L}_{\text{surf}} + \lambda_b \,\text{mean}_{v,c,k}(\hat{b}_{vck} - t_{vck})^2 + \lambda_{\text{bal}} K \sum_{k=1}^{K} \left(\bar{g}_k - \tfrac{1}{K}\right)^2 + \lambda_{\text{floor}} \sum_{k=1}^{K} [\text{ReLU}(\rho - p_k)]^2. \tag{20}$$

Here $\mathcal{L}_{\text{vol}}$ and $\mathcal{L}_{\text{surf}}$ are mean squared errors of the normalized field residual over the non-wall and the wall vertices, $\bar{g}_k$ is expert k's routing weight averaged over the batch and vertices, and p_k is its assigned target's share of the total squared target norm. We use $(\lambda_b, \lambda_{\text{bal}}, \lambda_{\text{floor}}) = (0.5, 0.01, 1.0)$ and $\rho = 0.02$.

4 Experiments

4.1 Simulation Setup

We evaluate our approach on the AirFRANS [1] dataset, generating anisotropic triangular working meshes with gmsh [33], each with approximately 4.9k vertices and refinement near the airfoil, and interpolate the reference fields onto them. Since, the data is two-dimensional, we embedded and additional third dimention at $z = 0$. We use the official $800/200$ split. Additionally, we employ a data augmentation technique by calculating 16 remesh alternatives. During training one of them is used at random every epoch.

Per-field relative L_2 and R^2 are evaluated in physical units. Volume and Surface metrics are mean squared errors of the physical prediction error divided by the per-field standard deviation of the physical targets over the training set, taken over the non-wall and the wall vertices respectively. We also report the relative error of the lift coefficient across test cases. All models are evaluated against interpolated targets on the same working meshes, so these numbers are not directly comparable with benchmark scores computed on the original meshes.

Our checkpoint is selected from 20 checkpoints by test-set error, whereas the baselines use validation-based selection. The model is trained on all 800 training cases, 80 cases are reserved for validation. Training uses AdamW [34] with a one-cycle schedule [35] for 400 epochs and one mesh per batch. Autocast runs in `bfloat16`, while the incidence and analyzer operations stay in `fp32`. Each run uses one NVIDIA RTX 4090 with 24 GB of memory. Full settings are given in section S3.

Table 1: Comparison on the official AirFRANS $800/200$ split. Every row is evaluated on the same ≈ 4.9k-vertex working meshes and evaluation metrics. Baselines use their full input-feature arms; training and checkpoint-selection differences are specified in section 4.1. *Mesh use* names how mesh structure reaches the prediction: *none* for a point-set model, *operator* when a learned Hodge Laplacian computes the fields, *router* when it computes only the mixing weights. C_L is the relative error of the lift over the 200 test cases and ρ_L its Spearman correlation with the reference.

model	params	mesh use	rel-$L_2 \downarrow$	$R^2 \uparrow$	Volume$\downarrow$	Surf$\downarrow$	$C_L \downarrow$	$\rho_L \uparrow$
Transolver [17]	6.00M	none	0.0436	0.9953	0.00265	**0.00143**	**0.0103**	**0.99989**
PointNet [36]	0.08M	none	0.0689	0.9904	0.00451	0.00440	0.0149	0.99965
Pointwise MLP	0.02M	none	0.1743	0.9404	0.03576	0.03969	0.0860	0.99338
PSD-star HodgeFormer [21]	0.47M	operator	0.0800	0.9885	0.01028	0.01048	0.0681	0.99883
Hodge-routed MoE (ours)	3.02M	router	**0.0300**	**0.9983**	**0.00140**	0.00164	0.0225	0.99988

4.2 Results

Table 1 compares the models on the official split. Our model has the lowest relative L_2 error and the lowest interior mean squared error of the five, 31% and 47% below Transolver at half its parameter count. Transolver has the lower error on the two wall quantities, by a factor of 1.14 on the surface error and 2.2 on the lift coefficient. The fourth row is a HodgeFormer surrogate, which keeps the learned Hodge Laplacian in the prediction path. The per-field breakdown is given in full in the supplementary material. Specifically, the proposed method demonstrates the lowest relative L_2 error on three of the four fields and the second lowest on u_y, and the largest difference is on the eddy viscosity. The aggregate is the mean over the four fields.

Figure 2 shows one test case at the reported checkpoint, with the reference, prediction, and absolute error for each field. Its per-field relative L_2 errors are 0.0047, 0.0260, 0.0281, and 0.0676, compared with test-set means of 0.0056, 0.0259, 0.0255, and 0.0629. Visible errors occur near the airfoil and along the turbulent-viscosity wake.

Figure 3 places the four models beside the CFD reference on a common colour scale. The models separate more on the turbulent viscosity than on the pressure: the mean relative L_2 over the 200 test cases is 0.063, 0.116, 0.144 and 0.155 on ν_t for (a)–(d), against 0.026, 0.030, 0.066 and 0.085 on p. The reference carries fine-scale structure in the ν_t wake, and the ordering of the panels on that field follows the ordering of those errors.

5 Discussion and Conclusion

In this work, we propose a Hodge-routed mixture of transformer experts for steady-PDE prediction on unstructured meshes. A learned Hodge Laplacian computes the routing weights, and a separate analyzer constructs multiscale supervision targets. The motivation behind the deployment of such a construct originates from the observation that a vanilla transformer architecture that utilizes the proposed embedding demonstrates encouraging results, reaching an accuracy comparable to the Transolver model. We hypothesize that a specialization of this construct into areas with similar spectral characteristics could improve local accuracy. Initially, traditional spectral processing was implemented to determine this distribution among the experts. The hypothesis to be tested was whether a construct with learnable Laplacians could substitute for the spectral decomposition of the mesh.

We employ the official AirFRANS dataset and dataset split and perform remeshing to represent the discretized domain using unstructured meshes. We compare our approach with Transolver [17] and compute mean relative L_2 and interior mean squared errors, as well as surface and relative lift-coefficient errors. We observe reduced mean relative L_2 and interior mean squared errors, together with improved prediction of turbulent viscosity. Since these results were measured on the working mesh, they do not establish accuracy at the original CFD resolution. Nevertheless, we consider the findings indicative of the potential for experts guided towards areas with similar spectral characteristics to improve accuracy.

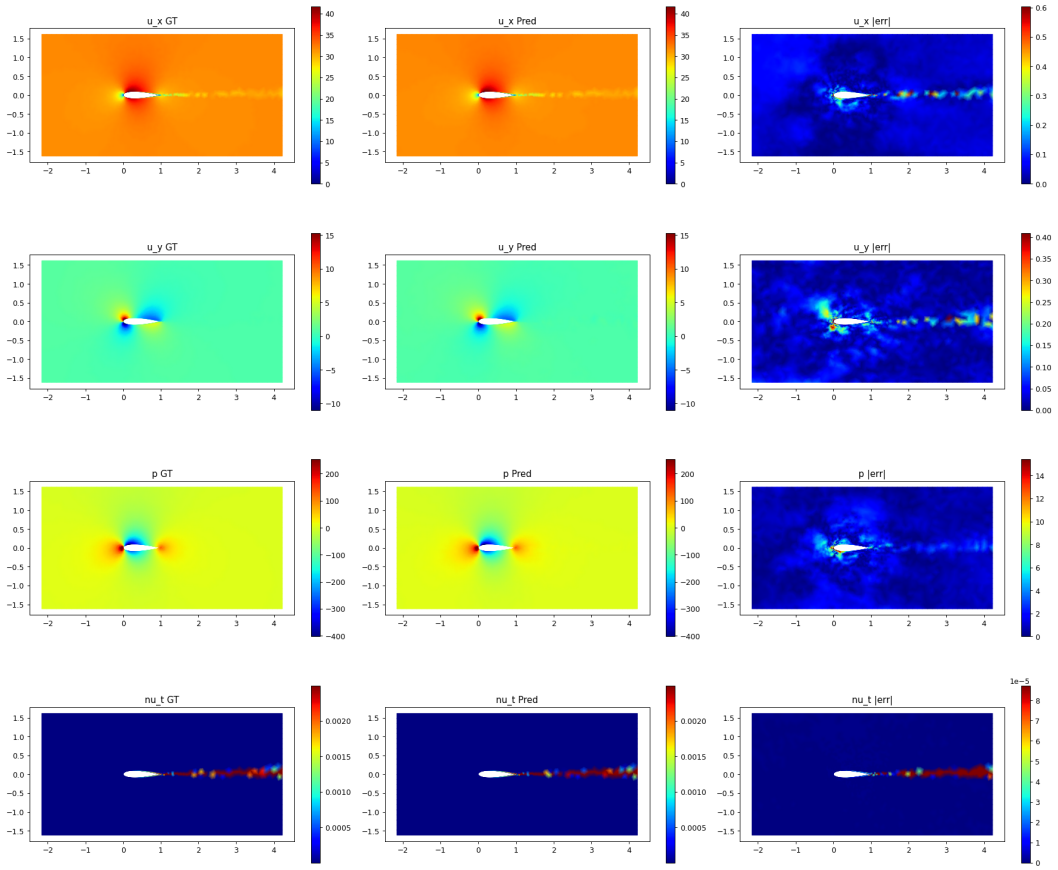
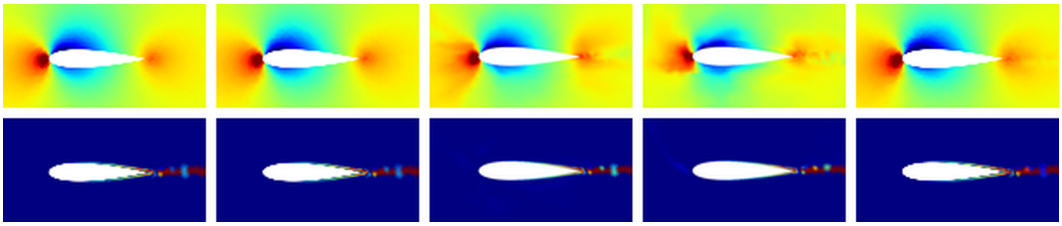


Figure 2: One test case at the reported checkpoint (epoch 380). Columns are the reference, the prediction and the absolute error; rows are u_x, u_y, p and ν_t. All panels are on the model's own working mesh in physical units, with the reference and the prediction sharing each field's colour scale. The per-field relative L_2 for this case is 0.0047, 0.0260, 0.0281 and 0.0676. Test-set means are listed in table S7; the panel illustrates one case rather than a test-set average.



CFD reference	**(a)**	**(b)**	**(c)**	**(d)**
—	rel-$L_2(p) = 0.026$ rel-$L_2(\nu_t) = 0.063$	rel-$L_2(p) = 0.030$ rel-$L_2(\nu_t) = 0.116$	rel-$L_2(p) = 0.066$ rel-$L_2(\nu_t) = 0.144$	rel-$L_2(p) = 0.085$ rel-$L_2(\nu_t) = 0.155$

Figure 3: Pressure p (top) and turbulent viscosity ν_t (bottom) on the reference colour scales; left to right, the CFD reference, (a) our model, (b) Transolver, (c) PointNet, and (d) the best PSD-star HodgeFormer. Values beneath each model panel are mean relative L_2 errors over the 200 test cases. The models differ more on ν_t than on p; the reference panel shows fine-scale structure in the ν_t wake.

References

[1] Florent Bonnet, Jocelyn Ahmed Mazari, Paola Cinnella, and Patrick Gallinari. AirFRANS: High fidelity computational fluid dynamics dataset for approximating Reynolds-averaged Navier–Stokes solutions. In *Adv. Neural Inform. Process. Syst.*, 2022. URL https://papers.neurips.cc/paper_files/paper/2022/hash/94ab7b23a345f93333eac8748a66c763-Abstract-Datasets_and_Benchmarks.html.

[2] Steven L. Brunton, Bernd R. Noack, and Petros Koumoutsakos. Machine learning for fluid mechanics. *Annual Review of Fluid Mechanics*, 52:477–508, 2020.

[3] Ricardo Vinuesa and Steven L. Brunton. Enhancing computational fluid dynamics with machine learning. *Nature Computational Science*, 2(6):358–366, 2022.

[4] Nils Thuerey, Konstantin Weißenow, Lukas Prantl, and Xiangyu Hu. Deep learning methods for Reynolds-averaged Navier–Stokes simulations of airfoil flows. *AIAA Journal*, 58(1):25–36, 2020.

[5] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.

[6] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021.

[7] Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with applications to PDEs. *J. Mach. Learn. Res.*, 24(89):1–97, 2023. doi: 10.5555/3648699.3648788.

[8] Kamyar Azizzadenesheli, Nikola Kovachki, Zongyi Li, Miguel Liu-Schiaffini, Jean Kossaifi, and Anima Anandkumar. Neural operators for accelerating scientific simulations and design. *Nature Reviews Physics*, 6:320–328, 2024. doi: 10.1038/s42254-024-00712-5. URL https://www.nature.com/articles/s42254-024-00712-5.

[9] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *Int. Conf. Learn. Represent.*, 2021. URL https://openreview.net/forum?id=c8P9NQVtmnO.

[10] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Graph kernel network for partial differential equations. *arXiv preprint arXiv:2003.03485*, 2020. URL https://arxiv.org/abs/2003.03485.

[11] Zongyi Li, Daniel Zhengyu Huang, Burigede Liu, and Anima Anandkumar. Fourier neural operator with learned deformations for PDEs on general geometries. *J. Mach. Learn. Res.*, 24(388):1–26, 2023. URL https://jmlr.org/papers/v24/23-0064.html.

[12] Tobias Pfaff, Meire Fortunato, Alvaro Sanchez-Gonzalez, and Peter W. Battaglia. Learning mesh-based simulation with graph networks. In *Int. Conf. Learn. Represent.*, 2021. URL https://openreview.net/forum?id=roNqYL0_XP.

[13] Shuhao Cao. Choose a transformer: Fourier or Galerkin. In *Adv. Neural Inform. Process. Syst.*, 2021.

[14] Zijie Li, Kazem Meidani, and Amir Barati Farimani. Transformer for partial differential equations' operator learning. *Trans. Mach. Learn. Res.*, April 2023. URL https://openreview.net/forum?id=EPPqt3uERT. arXiv:2205.13671.

[15] Zhongkai Hao, Zhengyi Wang, Hang Su, Chengyang Ying, Yinpeng Dong, Songming Liu, Ze Cheng, Jian Song, and Jun Zhu. GNOT: A general neural operator transformer for operator learning. In *Int. Conf. Mach. Learn.*, volume 202 of *Proceedings of Machine Learning Research*, pages 12556–12569, 2023. URL https://proceedings.mlr.press/v202/hao23c.html.

[16] Zijie Li, Dule Shu, and Amir Barati Farimani. Scalable transformer for PDE surrogate modeling. In *Adv. Neural Inform. Process. Syst.*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/590daf74f99ee85df3d8c007df9c8187-Paper-Conference.pdf. arXiv:2305.17560.

[17] Haixu Wu, Huakun Luo, Haowen Wang, Jianmin Wang, and Mingsheng Long. Transolver: A fast transformer solver for PDEs on general geometries. In *Int. Conf. Mach. Learn.*, volume 235 of *Proceedings of Machine Learning Research*, pages 53681–53705, 2024. URL https://proceedings.mlr.press/v235/wu24r.html.

[18] Mathieu Desbrun, Anil N. Hirani, Melvin Leok, and Jerrold E. Marsden. Discrete exterior calculus. *arXiv preprint math/0508341*, 2005.

[19] Dmitriy Smirnov and Justin Solomon. HodgeNet: Learning spectral geometry on triangle meshes. *ACM Trans. Graph.*, 40(4):1–11, 2021.

[20] Nicholas Sharp, Souhaib Attaiki, Keenan Crane, and Maks Ovsjanikov. DiffusionNet: Discretization agnostic learning on surfaces. *ACM Trans. Graph.*, 41(3):1–16, 2022.

[21] Akis Nousias and Stavros Nousias. HodgeFormer: Transformers for learnable operators on triangular meshes through data-driven hodge matrices. In *IEEE/CVF Winter Conf. on Applications of Computer Vision (WACV)*, pages 6796–6805, 2026. URL https://openaccess.thecvf.com/content/WACV2026/html/Nousias_HodgeFormer_Transformers_for_Learnable_Operators_on_Triangular_Meshes_through_Data-Driven_WACV_2026_paper.html. arXiv:2509.01839.

[22] Noam Shazeer et al. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *Int. Conf. Learn. Represent.*, 2017.

[23] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[24] Shizheng Wen, Arsh Kumbhat, Levi Lingsch, Sepehr Mousavi, Yizhou Zhao, Praveen Chandrashekar, and Siddhartha Mishra. Geometry aware operator transformer as an efficient and accurate neural surrogate for PDEs on arbitrary domains. *arXiv preprint arXiv:2505.18781*, 2025.

[25] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021. doi: 10.1038/s42256-021-00302-5.

[26] Zongyi Li, Nikola Kovachki, Chris Choy, Boyi Li, Jean Kossaifi, Shourya Otta, Mohammad Amin Nabian, Maximilian Stadler, Christian Hundt, Kamyar Azizzadenesheli, and Anima Anandkumar. Geometry-informed neural operator for large-scale 3D PDEs. In *Adv. Neural Inform. Process. Syst.*, 2023.

[27] Lek-Heng Lim. Hodge Laplacians on graphs. *SIAM Review*, 62(3):685–715, 2020.

[28] Michael T. Schaub, Austin R. Benson, Paul Horn, Gábor Lippner, and Ali Jadbabaie. Random walks on simplicial complexes and the normalized hodge 1-Laplacian. *SIAM Review*, 62(2):353–391, 2020. doi: 10.1137/18m1201019.

[29] Moshe Eliasof and Eran Treister. DiffGCN: Graph convolutional networks via differential operators and algebraic multigrid pooling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[30] Benjamin D. Shaffer, Shawn Koohy, Brooks Kinch, M. Ani Hsieh, and Nathaniel Trask. Structure-preserving learning improves geometry generalization in neural PDEs. In *Int. Conf. Mach. Learn.*, volume 306 of *Proceedings of Machine Learning Research*, pages 109470–109491, 2026. URL https://proceedings.mlr.press/v306/shaffer26a.html.

[31] Dongzhe Zheng, Tao Zhong, and Christine Allen-Blanchette. Topology-preserving neural operator learning via hodge decomposition. In *Int. Conf. Mach. Learn.*, volume 306 of *Proceedings of Machine Learning Research*, pages 164029–164060, 2026. URL https://proceedings.mlr.press/v306/zheng26aj.html.

[32] Lennart Bastian, Samuel Leventhal, Mustafa Hajij, and Tolga Birdal. Topological neural operators. *arXiv preprint arXiv:2606.09806*, 2026.

[33] Christophe Geuzaine and Jean-François Remacle. Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities. *Int. J. Numer. Methods Eng.*, 79(11):1309–1331, 2009.

[34] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *Int. Conf. Learn. Represent.*, 2019.

[35] Leslie N. Smith and Nicholay Topin. Super-convergence: Very fast training of neural networks using large learning rates. In *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, 2019.

[36] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. In *IEEE Conf. Comput. Vis. Pattern Recog.*, 2017.

Supplementary Material

S1 Notation

Table S1 defines the main symbols used in the paper and supplementary material. Array dimensions do not include the batch dimension. The symbols x_v, x_e, x_f denote input features in section 3.3 and normalized features used to compute the mesh operators.

In the embedding notation, $\delta_s = \mathrm{diag}(d_s^\top d_s)$ with $d_v = d_0$ and $d_e = d_1$, and A_s is the normalized adjacency matrix with its diagonal removed. In the HodgeFormer background (section S2.1), h counts attention heads and σ denotes softmax. Elsewhere, these symbols denote the shared embedding and Gaussian bandwidth.

Table S1: Notation. Main symbols grouped by their role. Array dimensions do not include the batch dimension. Uses specific to the HodgeFormer and transformer equations are identified.

symbol	type	meaning	symbol	type	meaning
The discretisation					
$\mathcal{M}$	(V, E, F)	the working mesh as a simplicial complex	$e = (i, j)$	index	an edge with endpoints i, j
V, E, F	sets	vertices, edges, faces	i, j, k, m, s	indices	elements i, j; expert k; atom m; stream s
N, N_e, N_f	integers	$\lvert V\rvert, \lvert E\rvert, \lvert F\rvert$; $N \approx 4900$	c	index	output channel in y_{vc}
v	index	a vertex			
Input features					
x_v	$\mathbb{R}^{N \times c_v}$	standardized vertex inputs	(c_v, c_e, c_f)	$(9, 21, 13)$	channel counts
x_e	$\mathbb{R}^{N_e \times c_e}$	edge geometry inputs, not standardized	pos	$\mathbb{R}^{N \times 2}$	standardized position columns of x_v
x_f	$\mathbb{R}^{N_f \times c_f}$	face geometry inputs, not standardized			
Target and prediction					
u_x, u_y	fields	velocity components	U_∞	scalar	freestream speed of a sample
p	field	kinematic pressure	y	$\mathbb{R}^{N \times 4}$	training target, standardised
ν_t	field	turbulent kinematic viscosity	$\hat{u}$	$\mathbb{R}^{N \times 4}$	standardized field prediction
Discrete operators					
d_0	incidence	$\mathbb{R}^{N_e \times N}$, $(d_0 x)_e = x_j - x_i$	L_0	$\mathbb{R}^{N \times N}$	vertex operator; stars depend on context
d_1	incidence	$\mathbb{R}^{N_f \times N_e}$, $d_1 d_0 = 0$	Q	$\mathbb{R}^{N \times N}$	controller matrix $d_0^\top \star_1 d_0$
$\star_0, \star_1, \star_2$	matrices	stars on 0-, 1-, 2-cochains	M	integer	atom count, $M = 9$
$\star_0^{-1}, \star_1^{-1}$	matrices	controller: independent learned operators			
Neighbourhoods and supports					
$\mathcal{N}^{\mathrm{full}}(i)$	set	stored neighbors: vertex/edge cap 256, face cap 1	support	pairs	the (i, j) with $j \in \mathcal{N}(i)$
$\mathcal{N}(i)$	set	effective neighbors; caps in table S3	mutual	property	$j \in \mathcal{N}(i) \Rightarrow i \in \mathcal{N}(j)$
n_{vcap}, n_{ecap}	$64, 32$	VEF vertex and edge caps	V, VEF	configurations	vertex-only; vertex, edge, and face updates

Table S1, continued

symbol	type	meaning	symbol	type	meaning
The controller (section 3.4)					
ℓ	$\mathbb{R}^{N\times K}$	vertex routing logits	$V^{(i)}$	array	projected values of head i; shape depends on stream
h_v, h_e, h_f	arrays	controller streams, each of width d_c	$W_V^{(i)}, W_{\text{out}}$	matrices	per-head value; per-layer output
d_c, L_c, H	$64, 2, 4$	width, layers, heads	W_a, W_b	matrices	read-out, $d_c \to 64 \to K$
d_h	16	d_c/H, per-head width	$\text{Emb}_s, \text{FFN}_s$	networks	embedding; feed-forward
A_s	matrix	normalized off-diagonal embedding adjacency	$\delta_s, \tilde{D}_s$	vector/matrix	incidence degrees; $\tilde{D}_s = \text{diag}(\delta_s) + I$
The Gaussian star (section 3.4)					
φ_i	$\mathbb{R}^{d_h}$	kernel feature, $W_\varphi \text{LN}(h)_i$	N_c	integer	element count in the feature array
W_φ	$\mathbb{R}^{d_h\times d_c}$	shared kernel projection for elements i, j	σ	> 0	bandwidth, $\text{softplus}(\gamma) + 10^{-8}$
w_{ij}	$(0, 1]$	Gaussian kernel weight	β	$(0, \frac{1}{2})$	$\frac{1}{2}\,\text{sigmoid}(\alpha)$
Ω	$\mathbb{R}^{N_c\times N_c}$	w_{ij} on the support, else 0	γ, α	scalars	raw parameters; init $0.5413, -0.85$
D, D_i	matrix/scalar	$D = \text{diag}(D_i)$; $D_i = \sum_j \Omega_{ij}$	A	matrix	unweighted vertex adjacency in fixed-band decomposition
$\hat{S}$	matrix	implemented normalization in eq. (6)			
Experts, gate and decoder (section 3.5)					
K	8	number of experts	W_{in}^k, W_b^k	matrices	expert input; band head
d, d_x	$192, 88$	token width; expert width	proj_k	projection	maps z_k to an $N \times d$ array, with bias
L_x, h_x	$2, 8$	blocks; heads per block	$\hat{b}_k$	$\mathbb{R}^{N\times 4}$	band-head output
h	$\mathbb{R}^{N\times d}$	token embedding, shared	$g(v)$	Δ^K	gate weights at vertex v
z_k, z_k^0	$\mathbb{R}^{N\times d_x}$	output, input of expert k	Δ^K	simplex	K nonnegative weights summing to one
n_F	8	frequencies; sine/cosine give $2n_F$ channels	h_{mix}	$\mathbb{R}^{N\times d}$	combined expert features
σ_k	constants	fixed Fourier scales, 1.0 to 38.0 (table S3)	E_{field}	$\mathbb{R}^{4\times d}$	decoder field embedding
G_k, B_k	$\mathbb{R}^{2\times n_F}$	fixed normal samples; $B_k = \sigma_k G_k$			
The analyzer (section 3.6)					
w_e	$\geq \epsilon_w$	edge conductance	$\bar{y}^{(j)}$	$\mathbb{R}^{N\times 4}$	target after m_j steps
ϵ_w	10^{-4}	keeps w_e positive	m_j, L	integers	m_j smoothing steps; $L = 8$ levels
μ_v	> 0	incident conductance sum; mass uses floor ϵ_w	a_i	$\mathbb{R}^{N\times 4}$	atom i; $\sum_i a_i = y$
MLP_e	$21 \to 64 \to 1$	edge-conductance score	P, θ	$\mathbb{R}^{K\times M}$	column-normalized assignment and its logits
τ	$\frac{1}{2}$	forward-Euler step	t_k	$\mathbb{R}^{N\times 4}$	band target; $\sum_k t_k = y$

Table S1, continued

symbol	type	meaning	symbol	type	meaning
The objective (section 3.7)					
$\mathcal{L}$	scalar	the training objective	$\bar{g}_k$	scalar	mean routing weight over vertices for expert k
$\mathcal{L}_{\text{vol}}, \mathcal{L}_{\text{surf}}$	scalars	interior, wall field terms	p_k	$[0, 1]$	assigned-target squared-norm share, denominator floored
Metrics					
rel-L_2	≥ 0	mean case/field relative L_2, physical units	C_L	≥ 0	relative L_2 of lift-coefficient vectors
R^2	≤ 1	coefficient of determination	ρ_L	$[-1, 1]$	Spearman correlation of lift
Volume, Surf	≥ 0	globally normalized MSE, non-wall and wall			
Generic operators					
LN	operator	layer normalisation	$\lambda(\cdot)$	set	the spectrum of a matrix
Drop	operator	dropout, probability 0.1	$\sim$	relation	similarity of matrices
GELU, MHSA	operators	activation; self-attention	$[\,\cdot\,]$	operator	concatenation; axis specified locally
$\text{diag}(\cdot)$	matrix	diagonal from a vector	$\odot$	operator	elementwise product
Additional local notation					
L_v, L_e, L_f	operators	vertex, edge, face operators in eq. (8)	B_{batch}	integer	batch size
x_v^{raw}	$\mathbb{R}^{N\times 9}$	vertex features before standardization	A_v^{dual}	area	barycentric dual vertex area
$\mu_c^{\text{norm}}, s_c^{\text{norm}}$	scalars	per-channel statistics of scaled training targets	$\widetilde{y}$	$\mathbb{R}^{N\times 4}$	freestream-scaled targets
T	1	assignment temperature in eq. (19)	E_k	scalar	$\sum_{v,c} t_{vck}^2$
$\lambda_b, \lambda_{\text{bal}}, \lambda_{\text{floor}}$	scalars	loss weights 0.5, 0.01, 1.0	ρ	0.02	minimum assigned-target norm share
e_{vc}	scalar	normalized field residual in eq. (20)	$V_{\text{vol}}, V_{\text{surf}}$	sets	non-wall and wall vertices
t_{vc}^r	vector	decoder field token at depth r	$m(v)$	$\mathbb{R}^d$	combined features after normalization and dropout
z_i	scalar	diagonal-face kernel score in eq. (7)			

S2 HodgeFormer Background

S2.1 HodgeFormer

HodgeFormer [21] extends the Transformer layer to operate on vertex, edge, and face features. Following the notation of the original paper, the feature arrays are

$$x_k \in \mathbb{R}^{n_k \times d_k}, \qquad k \in \{v, e, f\}. \tag{S1}$$

For a common embedding dimension $d_k = d$ and h attention heads, each head has dimension $d_h = d/h$. Learnable projection matrices define the query, key, and value representations,

$$\begin{aligned} Q_k &= x_k W_{Q_k}, \\ K_k &= x_k W_{K_k}, \qquad W_{Q_k}, W_{K_k}, W_{V_k} \in \mathbb{R}^{d \times d_h}. \\ V_k &= x_k W_{V_k}, \end{aligned} \tag{S2}$$

The original HodgeFormer parameterizes a Hodge Star through row-wise softmax attention,

$$\star_k(x_k) = \sigma\left(\frac{Q_k K_k^T}{\sqrt{d_h}}\right), \tag{S3}$$

where $\sigma(\cdot)$ denotes the row-wise softmax. Separate linear maps are used for the inverse Hodge stars. The learned operators are the following products of star and signed incidence matrices:

$$\begin{aligned} L_v &:= \star_0^{-1}(x_v) \cdot d_0^T \cdot \star_1(x_e) \cdot d_0, \\ L_e &:= d_0 \cdot \star_0^{-1}(x_v) \cdot d_0^T \cdot \star_1(x_e) \\ &\quad + \star_1^{-1}(x_e) \cdot d_1^T \cdot \star_2(x_f) \cdot d_1, \\ L_f &:= d_1 \cdot \star_1^{-1}(x_e) \cdot d_1^T \cdot \star_2(x_f). \end{aligned} \tag{S4}$$

For one head, the attention updates apply each operator to the corresponding projected values,

$$\begin{aligned} A_v(x_v, x_e) &= L_v(x_v, x_e) \cdot V_v(x_v), \\ A_e(x_v, x_e, x_f) &= L_e(x_v, x_e, x_f) \cdot V_e(x_e), \\ A_f(x_e, x_f) &= L_f(x_e, x_f) \cdot V_f(x_f). \end{aligned} \tag{S5}$$

The head outputs are joined along the feature dimension. A linear projection restores the embedding width. These operations define multi-head Hodge attention A_{H_l}. The HodgeFormer layer is

$$\begin{aligned} H_l(x_v, x_e, x_f) := G_l\big(&A_{H_l}(x_v, x_e, x_f) \\ &+ (x_v, x_e, x_f)\big), \end{aligned} \tag{S6}$$

where G_l applies separate two-layer feedforward networks to the vertex, edge, and face features. Each network uses a residual connection, which adds its input to its output. These equations describe the original HodgeFormer. The Gaussian-star controller used in this work is specified in section 3.4.

S3 Experimental Protocol

This section specifies the training, architecture, preprocessing, and checkpoint-selection procedures.

Table S2: Training configuration.

Optimizer	AdamW [34]; weight decay 10^{-4}
Initial training	400 epochs with OneCycleLR [35], updated after each batch.
Additional VEF training	Resume the model and optimizer from epoch 400 for 200 additional epochs. A new OneCycleLR schedule increases the learning rate from 8×10^{-6} to 2×10^{-4} over the first 20 additional epochs, then decreases it to approximately 8×10^{-10}.
Batch size	One mesh
Numerical precision	Mixed-precision training with `bfloat16`; incidence-matrix products and target decomposition computed in 32-bit floating point
Dropout probability	0.1
Initialization	Xavier uniform initialization for parameter arrays with at least two dimensions
Model saving and evaluation	Model saved every 20 epochs; 30 saved models evaluated for VEF and 20 for V
Hardware	One NVIDIA RTX 4090 with 24 GB memory

Input arrays and normalization

Each vertex has 9 input channels: two position coordinates, barycentric dual area, two inlet velocity components, signed distance to the airfoil, two wall-normal components, and a wall indicator. Barycentric dual area is one third of the summed areas of incident triangles.

Table S3: Architecture. Settings for VEF and V. The experts and controller use separate input networks. Arrows specify consecutive layer dimensions. Transformer layers apply normalization before both the attention and feedforward sublayers.

Component	Setting	Configuration
Inputs	Channels per element	9 per vertex, 21 per edge, and 13 per face
Expert input MLP	Layer dimensions	$9 \to 192 \to 192$, with GELU between the two linear layers (eq. (4))
Experts	Number	8
	Feature dimensions	88 features per vertex; linear input projection $208 \to 88$ and output projection $88 \to 192$
	Transformer layers	2 per expert; 8 attention heads with 11 features per head; feedforward hidden dimension 352
	Positional features	8 Fourier frequencies giving 16 sine/cosine features per expert; frequency scales $(1.0, 1.7, 2.8, 4.8, 8.0, 13.5, 22.6, 38.0)$
Controller	Input processing	Vertex and edge inputs aggregated using incidence matrices; separate MLPs map vertex, edge, and face inputs to 64 features per element
	Hodge layers	2 layers, with 64 features per vertex, edge, and face
	Attention and routing	4 attention heads; feedforward hidden dimension 256; MLP $64 \to 64 \to 8$ followed by softmax produces mixture weights
	Maximum list lengths	VEF: 64 vertex, 32 edge, and 1 face entries; V: 64 vertex and 64 edge entries
Analyzer	Edge-weight network	MLP with hidden dimension 64
	Target decomposition	Smoothing step size $1/2$; smoothed targets evaluated after $(2, 4, 6, 8, 12, 16, 24, 32)$ iterations to construct 9 components
	Vertex weights	Each diagonal weight in $\star_0$ equals the sum of the positive weights on incident edges
	Gradient updates	Target components treated as constants when learning their assignment to experts; analyzer receives no gradient updates
Decoder	Attention layers	2 layers attending across the four field representations at each vertex; 4 attention heads; feedforward hidden dimension 768
	Field representations	One learned 192-dimensional embedding per output field, added to the combined expert features
	Output MLP	The same $192 \to 64 \to 1$ MLP maps each field representation to a scalar prediction
Parameters	Total	3,106,490 (VEF); 3,023,962 (V)

Geometric edge and face features are computed after embedding the planar mesh in three dimensions with zero third coordinate. Each edge has 21 channels: 12 coordinates for its endpoints and the opposite vertices of its incident triangles, 3 components of the normalized mean endpoint mesh normal, and 6 pairwise distances between those four vertices. Entries for missing opposite vertices and their associated distances are zero at mesh boundaries. Each face has 13 channels: 9 ordered vertex coordinates, 3 mesh-normal components, and triangle area. Edge and face inputs are not standardized; these are the *raw* geometric features.

Targets are scaled as in eq. (3): velocity components and turbulent kinematic viscosity are divided by the case's freestream speed U_∞, and pressure by U_∞^2. Each scaled target channel and each vertex input channel are then standardized using fixed training-set means and standard deviations.

Neighbour-list construction

Neighbour lists are computed once for each mesh and reused during training and evaluation. Vertex and edge lists are limited to $m = 256$ entries. Candidates are obtained by breadth-first search with a maximum of $R = 8$ graph steps. Vertex adjacency is defined by mesh edges; two edges are adjacent if they share a vertex. Ring r consists of candidates at graph distance r. The stored list for element i is denoted by $\mathcal{N}^{\text{full}}(i)$. Ring 0 contains the element itself. Sampling quotas for rings $1, \dots, R$ are proportional to δ^{r-1}, with $\delta = 0.55$, and rounded to at least one entry per ring.

$$\mathcal{N}^{\text{full}}(i) \;\leftarrow\; \mathcal{N}^{\text{full}}(i) \cup \{\, j : i \in \mathcal{N}^{\text{full}}(j) \,\}, \tag{S7}$$

S4 The Model Family

Table S4 describes the proposed configuration and the compared models.

Mesh-free predictors. The internal plain-transformer variants read the features described in section 3.3.

Specifically, the *plain transformer* uses four pre-normalized softmax transformer layers of width 64 with global vertex attention, without incidence operators or taking into account the local neighborhood.

The *mesh-windowed* variant restricts attention to the geodesic-ring support, and the *locality-biased* variant adds a learned distance bias to global attention logits. The *wide plain transformer* uses width 192. The + *lift loss* variant adds a differentiable lift term to its objective.

Frequency mixtures of experts with an MLP router. An input MLP maps each vertex's input channels to a feature vector. This vector is passed to all transformer experts and to an MLP router, which computes one mixture weight per expert. These weights combine the experts' output features before field decoding. Each expert also predicts a target component through an auxiliary head.

The *fixed-band MoE* uses four experts, supervised by four target components obtained by fixed graph smoothing and differences between smoothing levels. The two *entropy-balanced* variants instead construct these components with a Hodge analyzer. An entropy term encourages a more even distribution of their squared norms. In the first variant, both expert supervision and the entropy term train the analyzer; in the detached variant, only the entropy term does.

The *learned-band MoE* uses eight experts and learns how to combine the analyzer's components into a target for each expert (eq. (19)). The components are frozen before assignment, so the analyzer is not updated during training. A penalty applies when an expert target's share of the summed squared norms falls below a certain threshold. The + *wall-surface branch* adds surface-mask and arc-length inputs, a transformer that predicts corrections on the airfoil surface, and a lift-weighted surface loss.

Hodge-routed mixtures. The proposed *Hodge-routed MoE (VEF)* replaces the learned-band MoE's MLP router with a PSD-Hodge network. Two Hodge layers update features on mesh vertices, edges, and faces. The router then uses the resulting vertex features to compute one mixture weight per expert at each vertex. The experts, target assignment, and field decoder use the learned-band design. The *vertex-only* variant (V) applies Hodge updates only to vertex features; edge and face features still receive MLP updates. Its *wide-expert* variant increases the widths of the input embedding, experts, and decoder, giving 5.68M parameters.

Hodge predictors. These models use Hodge layers to compute the features from which the flow fields are predicted, following HodgeFormer [21]. The *PSD-star HodgeFormer* uses learned Gaussian-kernel Hodge operators, one global transformer block, and a separate MLP output head for each field. The + *cross-field decoder* variant replaces these heads with an attention decoder that combines information across fields, increases the global transformer blocks from one to two, and normalizes training errors by each field's standard deviation within each case. The + *disjoint surface loss* variant also adds a wall-indicator input and computes separate losses on wall and non-wall vertices (eq. (20)).

External baselines. Transolver [17], PointNet [36], and a pointwise MLP are trained on vertex data from the working meshes and evaluated using the same metric definitions.

Table S4: Ablation studies.

Model	Architecture and training configuration	Parameters
Transformer predictors		
Plain transformer	4 global self-attention layers with pre-layer normalization; embedding dimension 64	0.31M
windowed	Self-attention restricted to mesh neighbourhoods defined by geodesic rings	0.31M
+ locality bias	Global self-attention with a learned distance-based attention bias	0.31M
Wide plain transformer	Embedding dimension 192, feedforward dimension 768, 12 attention heads	2.72M
+ lift loss	Additional loss on C_L computed from the predicted pressure	2.72M
Frequency MoE with MLP routing		
Fixed-band MoE	4 transformer experts; MLP routing; auxiliary targets from fixed graph smoothing	2.75M
entropy-balanced	Hodge analyzer trained by auxiliary prediction loss and entropy maximization of target squared-norm shares	2.75M
entropy-balanced, detached	Expert supervision uses detached targets; analyzer optimized only by entropy maximization	2.75M
Learned-band MoE	8 experts; learned target assignment; detached analyzer outputs; penalty on target squared-norm shares below a threshold	2.75M
+ wall-surface branch	Surface-mask and arc-length inputs; surface transformer; lift-weighted surface loss	3.00M
Hodge-routed MoE		
Hodge-routed MoE (VEF)	Proposed router: vertex, edge, and face Hodge updates; mixture weights computed from vertex features	3.11M
Vertex-only controller (V)	Vertex Hodge updates; edge and face features updated by MLPs	3.02M
wide experts (V)	Embedding and decoder dimensions 256; expert dimension 128; vertex-only controller	5.68M
Hodge predictors		
PSD-star HodgeFormer	Learned Gaussian-kernel Hodge operators; global transformer; separate MLP output heads for each field	0.45M
+ cross-field decoder	Attention decoder across fields; two global transformer blocks; fieldwise normalization of loss within each case	0.47M
+ disjoint surface loss	Separate MSE terms on wall and non-wall vertices; additional wall-indicator input	0.47M
symmetric support	Gaussian kernels restricted to reciprocal neighbour pairs	0.47M
no vertex star	Vertex star omitted: $L_0 = d_0^\top \star_1 d_0$	0.47M
relaxed β bound	Star mixing coefficient β allowed to exceed $1/2$	0.47M
dot-product kernel	Gaussian kernel replaced by dot-product softmax attention on the same neighbourhoods	0.47M
1 / 4 layers	Hodge layer count changed from 2 to 1 or 4	0.35/0.72M
External baselines		
Transolver [17]	Self-attention between learned slice tokens, followed by projection to vertices	6.00M
PointNet [36]	Pointwise MLPs with global max pooling	0.08M
Pointwise MLP	Pointwise multilayer perceptron with batch normalization	0.02M

S5 Effect of Input Features on Transolver

We evaluate the Transolver architecture taking into account different variants of input features.

Table S5: Transolver variants with different input features. Results on the 200 official test cases. Volume and Surf are normalized mean squared errors on non-wall and wall vertices, respectively. Arrows indicate whether lower or higher values are preferred; the best value for each metric is in bold.

Input features	Channels	Parameters	rel-L_2 ↓	R^2 ↑	Volume↓	Surf↓	C_L ↓	ρ_L ↑
Position, inlet velocity	4	5,995,332	0.0439	**0.9954**	0.00270	0.00140	**0.0078**	0.99988
Position, inlet velocity, area	5	5,995,844	0.0592	0.9922	0.00418	0.00282	0.0115	0.99980
Position, inlet velocity, distance, normals	7	5,996,868	0.0440	0.9952	0.00286	**0.00137**	0.0098	0.99977
All listed features	8	5,997,380	**0.0436**	0.9953	**0.00265**	0.00143	0.0103	**0.99989**

S6 Complete Results

Overall prediction errors

Table S6 compares the models on the same 200 test meshes, each with approximately 4.9k vertices. Volume and Surf are normalized mean squared errors on non-wall and wall vertices, respectively.

The reported metrics refer to checkpoints corresponding to lowest mean relative L_2 error on the test set. Transolver, PointNet, and the pointwise MLP use checkpoints selected by validation error.

For the external baselines, *all inputs* denotes position, dual area, inlet velocity, signed distance to the airfoil, and wall normals. The Transolver variant labelled *no dual area* receives all of these features except dual area.

Table S6: Overall prediction errors. Metrics at each model's selected checkpoint. Mesh role indicates the use of connectivity: Routing computes expert mixture weights; Targets constructs auxiliary training targets; Prediction computes features used to predict the fields; None indicates that connectivity is not used. The best value for each metric is in bold.

Model	Mesh role	Parameters	rel-L_2 ↓	R^2 ↑	Volume↓	Surf↓	C_L ↓
Hodge-routed MoE (VEF)	Routing	3.11M	0.0359	0.9978	0.00182	0.00180	0.0277
Hodge-routed MoE (V)	Routing	3.02M	**0.0300**	**0.9983**	**0.00140**	0.00164	0.0225
wide experts (V)	Routing	5.68M	0.0341	0.9979	0.00163	0.00219	0.0286
Learned-band MoE	Targets	2.75M	0.0345	0.9979	0.00165	0.00192	0.0265
+ wall-surface branch	Targets	3.00M	0.0378	0.9973	0.00196	0.00227	0.0247
Fixed-band MoE	Targets	2.75M	0.0351	0.9978	0.00178	0.00184	0.0280
entropy-balanced, detached	Targets	2.75M	0.0402	0.9973	0.00206	0.00214	0.0270
entropy-balanced	Targets	2.75M	0.0412	0.9972	0.00198	0.00233	0.0322
Transolver, all inputs [17]	None	6.00M	0.0436	0.9953	0.00265	0.00143	0.0103
Transolver, position + inlet	None	6.00M	0.0439	0.9954	0.00270	0.00140	**0.0078**
Transolver, no dual area	None	6.00M	0.0440	0.9952	0.00286	**0.00137**	0.0098
Wide plain transformer	None	2.72M	0.0446	0.9960	0.00265	0.00192	0.0245
+ lift loss	None	2.72M	0.0458	0.9960	0.00248	0.00296	0.0378
Plain transformer	None	0.31M	0.0544	0.9943	0.00374	0.00300	0.0173
PointNet, all inputs [36]	None	0.08M	0.0689	0.9904	0.00451	0.00440	0.0149
Pointwise MLP, all inputs	None	0.02M	0.1743	0.9404	0.03576	0.03969	0.0860
PSD-star HodgeFormer, + surface loss	Prediction	0.47M	0.0800	0.9885	0.01028	0.01048	0.0681
PSD-star HodgeFormer, + decoder	Prediction	0.47M	0.0822	0.9884	0.00803	0.01205	0.0567
PSD-star HodgeFormer	Prediction	0.45M	0.1154	0.9769	0.01446	0.03174	0.1747

Errors for individual fields

Table S7 reports relative L_2 errors separately for the two velocity components u_x and u_y, pressure p, and turbulent kinematic viscosity ν_t. Table S8 reports normalized mean squared errors for each field, separated into non-wall vertices (Volume) and wall vertices (Surf). Both tables use the same selected checkpoints as table S6.

Checkpoint selection

Table S9 lists the lowest mean relative L_2 and Volume errors among the evaluated checkpoints.

S7 Target Decomposition and Routing Weights

The following figures show the target decomposition and expert mixture weights of the vertex-only model (V) for one test mesh.

The analyzer splits the target fields into nine components using repeated graph smoothing and differences between smoothing levels. These components, labelled *atoms* in fig. S1, sum to the original fields. The first component is the most-smoothed field; the last is the residual after the fewest smoothing steps. The analyzer's edge weights depend on mesh geometry and remain at their

Table S7: Relative L_2 error for each predicted field. Values at the selected checkpoint of each model. The lowest error in each column is in bold.

Model	u_x	u_y	p	ν_t
Hodge-routed MoE (VEF)	0.0067	0.0348	0.0380	0.0642
Hodge-routed MoE (V)	**0.0056**	0.0259	**0.0255**	**0.0629**
wide experts (V)	0.0072	0.0319	0.0337	0.0638
Learned-band MoE	0.0068	0.0288	0.0365	0.0658
+ wall-surface branch	0.0064	0.0293	0.0342	0.0812
Fixed-band MoE	0.0068	0.0312	0.0346	0.0679
Transolver, all inputs	0.0067	**0.0220**	0.0298	0.1158
Wide plain transformer	0.0059	0.0330	0.0326	0.1068
Plain transformer	0.0067	0.0554	0.0349	0.1205
PSD-star HodgeFormer, + surface loss	0.0149	0.0650	0.0852	0.1548
PointNet, all inputs	0.0125	0.0540	0.0655	0.1435
Pointwise MLP, all inputs	0.0518	0.1865	0.1765	0.2825

Table S8: Normalized mean squared error by field and region. Volume uses non-wall vertices; Surf uses wall vertices. All values use each model's selected checkpoint.

	Volume (non-wall)				Surf (wall)			
Model	u_x	u_y	p	ν_t	u_x	u_y	p	ν_t
Hodge-routed MoE (VEF)	0.00019	0.00096	0.00077	0.00534	0.00079	0.00197	0.00272	0.00173
Hodge-routed MoE (V)	0.00011	0.00052	0.00031	0.00466	0.00073	0.00174	0.00248	0.00160
Learned-band MoE	0.00019	0.00063	0.00072	0.00506	0.00087	0.00195	0.00307	0.00181
Fixed-band MoE	0.00019	0.00086	0.00069	0.00539	0.00090	0.00198	0.00296	0.00152
Wide plain transformer	0.00017	0.00109	0.00074	0.00859	0.00062	0.00191	0.00313	0.00201
Plain transformer	0.00019	0.00279	0.00060	0.01140	0.00088	0.00403	0.00431	0.00278
PSD-star HodgeFormer + surface loss	0.00114	0.00476	0.00420	0.03103	0.00406	0.00609	0.01801	0.01378

Table S9: Minimum errors across evaluated checkpoints. Each column reports the minimum of that metric, with its epoch in parentheses. VEF was trained for 600 epochs, including 200 additional epochs with a peak learning rate of 2×10^{-4}.

Model	Checkpoints	Minimum rel-L_2 (epoch)	Minimum Volume (epoch)
Hodge-routed MoE (VEF)	30	0.0359 (560)	0.00182 (560)
Hodge-routed MoE (V)	20	0.0300 (380)	0.00139 (400)
wide experts (V)	20	0.0341 (400)	0.00163 (400)
Learned-band MoE	20	0.0345 (380)	0.00165 (380)
Fixed-band MoE	20	0.0351 (400)	0.00178 (380)
Plain transformer	20	0.0544 (320)	0.00339 (400)
PSD-star HodgeFormer, + surface loss	20	0.0800 (320)	0.00651 (400)

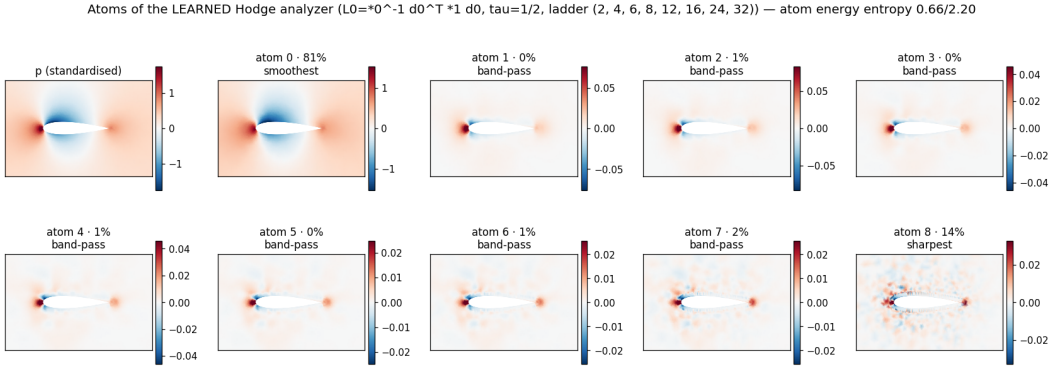


Figure S1: Target components produced by the analyzer. Maps show pressure and its nine components. Component 0 is the most-smoothed field, components 1–7 are differences between smoothing levels, and component 8 is the remaining residual. The displayed percentages are shares of the summed squared component norms across all four standardized fields on this mesh.

initialized values during training. The weights assigning these components to experts are learned during training.

Figure S2 shows how the nine components are combined into targets for the eight experts. Each column of the assignment matrix contains the weights used to distribute one component among the experts. The column weights sum to one, so the eight assigned targets also sum to the original fields. The bar chart compares component magnitudes before and after this assignment, using squared norms summed over all four standardized fields.

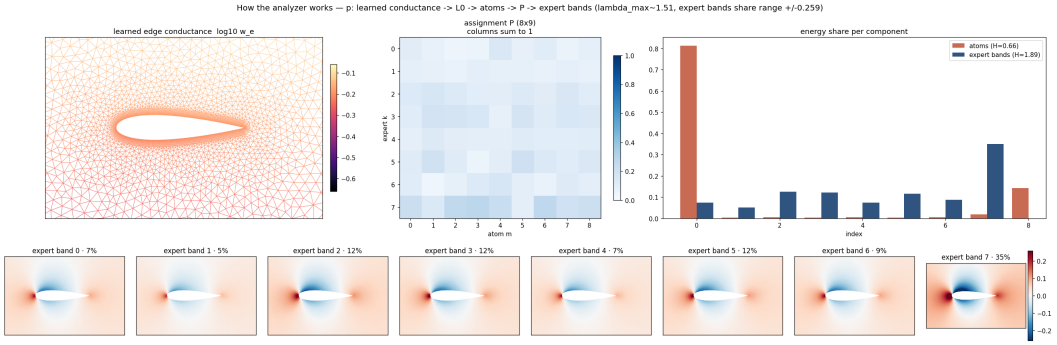


Figure S2: Assignment of target components to experts. Top: the analyzer's fixed edge weights, the learned 8×9 assignment matrix, and squared-norm shares before and after assignment. Bottom: the eight assigned pressure targets on a common colour scale. Norm shares are computed across all four standardized fields on this mesh.

The router computes eight mixture weights at each vertex to combine the experts' output features. Figure S3 shows the expert with the largest weight, the entropy of the weight distribution, and the mean weight of each expert over the mesh. Higher entropy indicates a more even distribution of weights among experts. The mean weights are close to the uniform value $1/8$, while the individual weight maps show spatial variation. All eight experts contribute through their mixture weights.

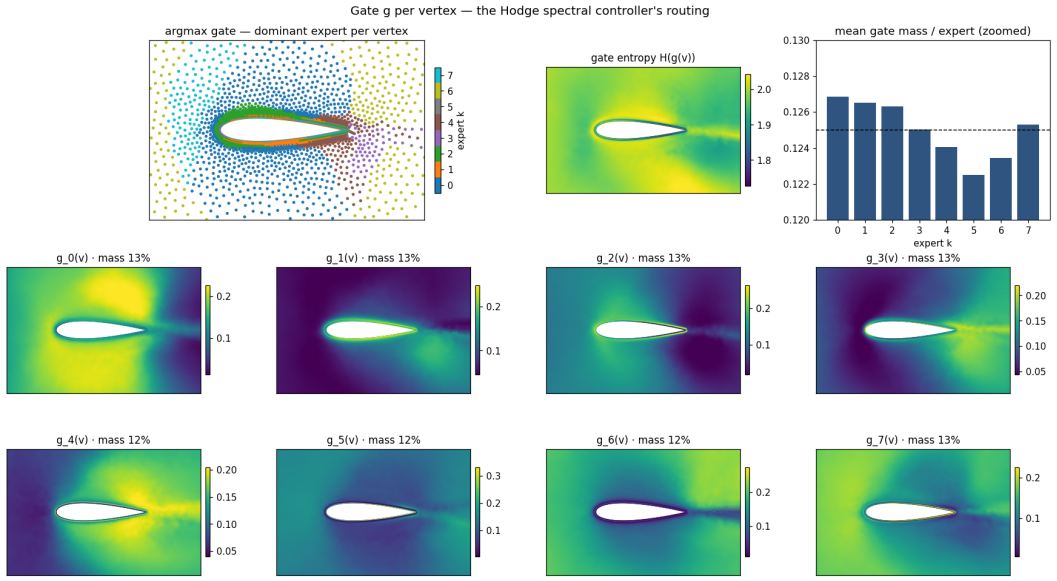


Figure S3: Expert mixture weights on the test mesh. Top: the expert with the largest weight at each vertex, the entropy of the eight weights, and each expert's mean weight over vertices. The dashed line denotes uniform weighting, $1/8$; the bar-chart axis spans 0.120–0.130. Bottom: the spatial distribution of each expert's mixture weight.

Figure S4 shows a fixed graph-smoothing decomposition of the same target fields. Smoothing after 2, 8, and 32 steps defines four components that sum to the original fields. The maps display the pressure components, with percentages computed across all four standardized fields as in the analyzer figures above.

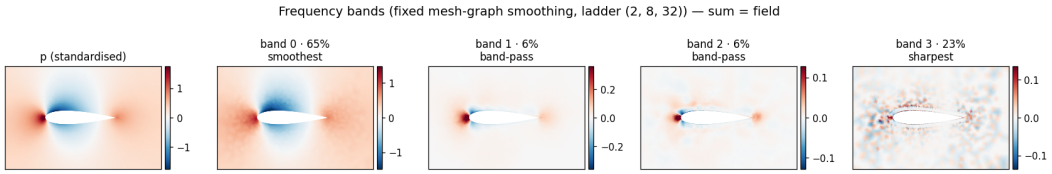


Figure S4: Target components from fixed graph smoothing. Pressure and four components constructed using prescribed smoothing steps. The percentages give each component's share of the summed squared norms across all four standardized fields on this mesh.